# Nodo Pañol
# Conexionado, pines y lógica

Control de acceso a pañol con RFID, solenoide, reed switch y detector de presencia, sobre ESP32. Documento de referencia para el maquetado y armado del prototipo.

**NODO**
`panol-lab01-puerta`

**UBICACIÓN**
`panol-lab01`

**MICROCONTROLADOR**
ESP32-WROOM-32 · MicroPython 1.28

**ESTADO**
Validado sobre hardware real (banco)

Todos los datos de pines y tiempos de este documento están tomados directamente del firmware en producción (`firmware/nodo_panol/config.py` y `main.py`). Ante cualquier diferencia con documentación anterior, **manda el código**. Ver §12, «Puntos abiertos».

# 01 Qué hace el nodo

El nodo controla la puerta del pañol: lee una tarjeta RFID, decide localmente si está autorizada, pulsa un solenoide para destrabar, y confirma con un reed switch que la puerta realmente se abrió. Todo lo que pasa lo reporta a un servidor por WiFi.

## El principio de diseño, en una frase

**La red nunca está en el camino crítico de abrir la puerta.** El nodo decide contra una lista blanca guardada en su propia flash y *después* reporta. Si se cae el WiFi, el servidor o la luz del edificio, la puerta sigue funcionando para quien tiene tarjeta, y los eventos se guardan en un archivo hasta que haya red de nuevo.

## La cadena de responsabilidades

| CAPA | MECANISMO | QUÉ GARANTIZA ANTE UN FALLO TOTAL |
|---|---|---|
| **1 · Mecánica** | Picaporte interior siempre operativo | Evacuación siempre posible. No depende de nada eléctrico. |
| **2 · Eléctrica** | Solenoide *fail-secure* + resistencia de estado seguro en la compuerta | Sin energía la puerta queda **trabada desde afuera**, y sin pulsos espurios al arrancar. |
| **3 · Firmware** | FSM volátil + watchdog + cola en flash | Un reinicio nace en estado seguro; no se pierden eventos de auditoría. |
| **4 · Servidor** | PostgreSQL como única fuente de verdad | La sesión y las alarmas sobreviven al apagón. |

> **REGLA DE ORO**
>
> La capa que te salva en un corte de luz debe ser la **más tonta** (mecánica), no la más lista (software). Si todo se apaga, el sistema falla en el estado seguro por física, no por lógica.

## Qué NO hace el nodo

No maneja sesiones ni sabe quién está adentro. No interpreta el reed ni el PIR: los reporta como eventos sueltos y el servidor decide qué significan. Esa separación es deliberada — si el nodo se resetea a mitad de una apertura, el servidor nunca queda con una sesión a medias.

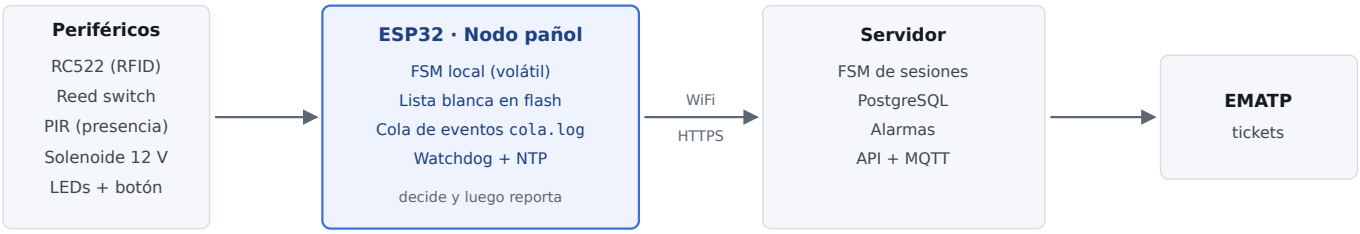



**Figura 1.** El nodo es solo el extremo físico del sistema. Este documento cubre únicamente la caja azul y sus periféricos.

## 02 Lista de componentes

| COMPONENTE | MODELO USADO | ALIMENTACIÓN | NOTAS PARA EL MAQUETADO |
|---|---|---|---|
| **Microcontrolador** | ESP32-WROOM-32 (devkit) | 5 V USB | **WROOM, no WROVER.** En el WROVER el GPIO16 lo ocupa la PSRAM y ahí va el PIR. |
| **Lector RFID** | MFRC522 (módulo RC522) | 3,3 V | VCC **exclusivamente** a 3,3 V. A 5 V se quema el chip. Bus SPI. |
| **Sensor de puerta** | Reed switch NA + imán | — | Contacto seco. Sin alimentación: usa el pull-up interno del ESP32. |
| **Detector de presencia** | DSC LC-100-PI | 12 V | **No es un HC-SR501.** Salida por relé de contacto seco N.C. Requiere fuente de 12 V y masa común. |
| **Cerradura** | Solenoide *fail-secure* | 12 V | Energizar-para-abrir. Sin corriente queda trabado, que es lo correcto para acceso controlado. |
| **Driver de potencia** | MOSFET N de nivel lógico (p. ej. IRLZ44N / IRL540N) | — | **De nivel lógico** ($V_{GS(th)}$ bajo): el ESP32 solo entrega 3,3 V de compuerta. Un IRF540 común no satura. |
| **Protección de bobina** | Diodo 1N4007 | — | **Obligatorio**, en antiparalelo con la bobina. Sin él, el primer corte destruye el MOSFET. |
| **Resistencias** | 10 kΩ · 220 Ω · 220–330 Ω ×2 | — | Estado seguro de compuerta · resistencia de compuerta · limitadoras de LED. |
| **Indicadores** | 2 LED (acceso y puerta) | — | El de acceso puede ser el LED integrado de la placa (GPIO2). |
| **Botón** | Pulsador NA | — | Alta manual de tarjetas. Con resistencia de *pull-down*: en reposo tiene que leer 0. |
| **Fuente** | Switching 12 V · ≥ 2 A | 12 V | Alimenta solenoide y PIR. El pico de arranque del solenoide es lo que fija la corriente. |

**SOBRE EL MOSFET**

El circuito de potencia se puede armar con un **módulo de relé** en vez de MOSFET, pero entonces hay que revisar la polaridad: la mayoría de los módulos de relé chinos son **activo en LOW**, mientras que el firmware actual activa en **HIGH** (ver §6.4 y §12). Con MOSFET N en configuración *low-side* —lo que asume este documento— activo en HIGH es lo natural.

## 03 Mapa de pines

Ésta es la tabla autoritativa. Sale de `firmware/nodo_panol/config.py`. Cualquier otro mapa de pines que circule está desactualizado.

| FUNCIÓN | GPIO | MODO | DETALLE |
|---|---|---|---|
| **LECTOR RFID — MFRC522 (SPI2 / VSPI)** | | | |
| RC522 · SCK | **GPIO18** | SPI CLK | Reloj del bus. |
| RC522 · MOSI | **GPIO23** | SPI MOSI | ESP32 → RC522. |
| RC522 · MISO | **GPIO19** | SPI MISO | RC522 → ESP32. Si este cable falla, el lector «no responde». |
| RC522 · RST | **GPIO4** | Salida | **Obligatorio cablearlo.** El driver hace un reset por hardware; sin este pin el chip queda dormido. |
| RC522 · SDA / CS | **GPIO5** | Salida | Chip select, activo en LOW. |
| RC522 · IRQ | – | — | **No se cablea.** El driver funciona por sondeo. |
| **ACTUADOR** | | | |
| Compuerta del MOSFET (solenoide) | **GPIO15** | Salida | **Pin de strapping.** Requiere resistencia externa de estado seguro. Pulso de 800 ms. Ver §6.4. |
| **SENSORES** | | | |
| Reed switch | **GPIO27** | Entrada + PULL_UP interno | Una pata al pin, la otra a GND. **0 = cerrada · 1 = abierta.** |
| PIR (contacto seco N.C.) | **GPIO16** | Entrada + PULL_UP interno | C → GND, NC → este pin. **0 = reposo · 1 = movimiento.** |
| **INTERFAZ DE USUARIO** | | | |
| LED de acceso | **GPIO2** | Salida | LED integrado de la placa. Encendido mientras dura la ventana de apertura. Pin de strapping. |
| LED de puerta | **GPIO33** | Salida | Encendido = puerta abierta (espejo del reed). |
| Botón «grabar tarjeta» | **GPIO32** | Entrada + PULL_DOWN interno | En reposo **tiene que leer 0**. Si al arrancar lee 1, el firmware desactiva la función. |

> **CORRECCIÓN RESPECTO DE DOCUMENTACIÓN PREVIA**
>
> El `README.md` del repositorio todavía dice **GPIO26** para el solenoide y **HC-SR501** para el PIR. Ambos datos son viejos: el pin se movió a **GPIO15** y el sensor real es un **DSC LC-100-PI** de contacto seco. Usar esta tabla, no la del README.

## 04 Diagrama de conexionado

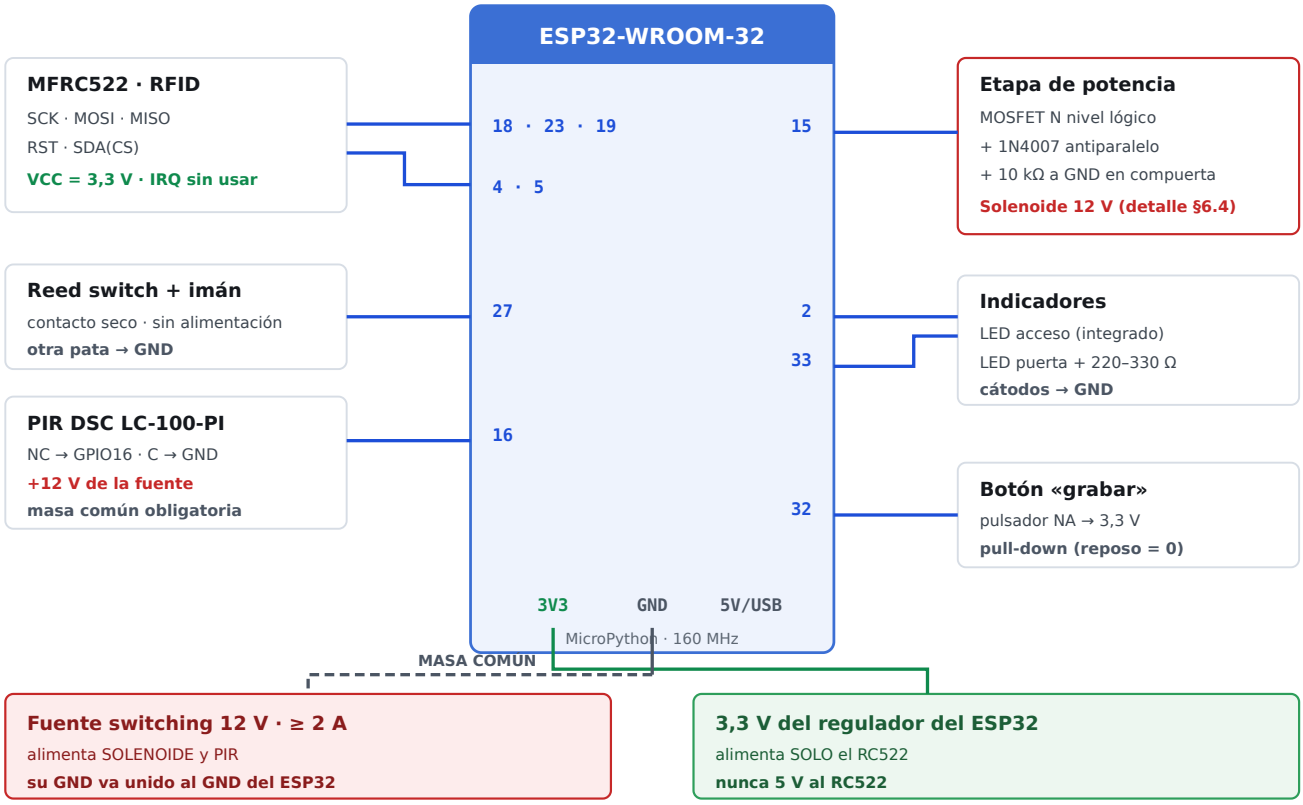



**Figura 2.** Conexionado general. Las líneas azules son señal lógica a 3,3 V; la línea punteada gris es la masa común entre la fuente de 12 V y el ESP32, que es obligatoria para que el PIR y el MOSFET funcionen. La etapa de potencia se detalla en la figura 3.

**LOS TRES ERRORES QUE MATAN LA PLACA**

**1.** RC522 alimentado a 5 V → se quema el chip, sin vuelta atrás.
**2.** Solenoide sin diodo 1N4007 → el primer corte de la bobina destruye el MOSFET.
**3.** Masa de los 12 V no unida a la del ESP32 → el PIR y el MOSFET no conmutan, o conmutan de manera errática, y se pierde tiempo buscando el problema en el software.

## 05 Alimentación y masas

Hay tres tensiones en juego y una sola masa. Confundirlas es la fuente más común de fallos intermitentes.

| RIEL | ORIGEN | ALIMENTA | CUIDADOS |
|---|---|---|---|
| 12 V | Fuente switching externa (≥ 2 A) | Solenoide · PIR DSC | Nunca toca un GPIO. El pico de arranque del solenoide es lo que fija el amperaje de la fuente. |
| 5 V | USB o pin VIN del devkit | Solo el propio ESP32 | No alimentar el RC522 desde acá. |
| 3,3 V | Regulador de la placa ESP32 | RC522 · pull-ups · botón | El regulador del devkit entrega poco (≈ 500 mA). Con el RC522 alcanza sobrado. |
| GND | Único, compartido | Todo | **Obligatorio:** el negativo de los 12 V debe unirse al GND del ESP32. |

**POR QUÉ LA MASA COMÚN NO ES OPCIONAL**

El PIR conmuta un **contacto seco** y el MOSFET compara su tensión de compuerta **contra su fuente (source)**. Si las masas están separadas, no existe una referencia común: el ESP32 «ve» un potencial flotante en vez de un 0 firme. El síntoma típico son lecturas erráticas del PIR o un solenoide que a veces dispara y a veces no. Se verificó en el banco que, con la masa bien unida, el PIR entrega pulsos limpios de ~1,7 s sin rebotes.

### Consumo y dimensionamiento

El solenoide es la única carga significativa, y trabaja por **pulsos de 800 ms**, no de forma continua. Aun así conviene dimensionar la fuente por el pico de arranque y no por el promedio: un solenoide típico de 12 V pide entre 0,5 A y 1,5 A durante ese pulso. Con 2 A hay margen para el PIR (consumo despreciable) y para que la tensión no se hunda y reinicie el ESP32.

**SUGERENCIA PARA EL MAQUETADO**

Si en la maqueta el ESP32 se reinicia justo cuando dispara el solenoide, casi seguro es una caída de tensión por la fuente o por cables finos, no un bug del firmware. Un capacitor electrolítico de 470–1000 µF en el riel de 12 V, cerca del solenoide, suele resolverlo.

# 06 Los subsistemas, uno por uno

## 6.1 · Lector RFID MFRC522

| | |
|---|---|
| Bus | SPI de hardware (SPI2 / VSPI) a 1 MHz, modo 0 (polaridad 0, fase 0) |
| Pines | `SCK 18 · MOSI 23 · MISO 19 · RST 4 · SDA/CS 5` |
| Alimentación | **3,3 V exclusivamente** |
| Registro de versión | reg. `0x37` debe leer `0x91` o `0x92` |
| UID leído | 4 bytes, formateados como `DE:AD:BE:EF` |

Al arrancar, el firmware reintenta la verificación del lector durante **20 segundos**, a propósito: da tiempo a acomodar los cables sin tener que resetear la placa.

> **DIAGNÓSTICO DEL RC522 — LO QUE YA NOS PASÓ**
>
> Si el registro `0x37` lee `0x00` o `0xFF`, el lector no está hablando. Las dos causas reales que encontramos:
>
> - **MISO mal cableado.** Es el único cable que va del módulo hacia el ESP32; si está flojo, el ESP32 escribe bien pero nunca recibe respuesta.
> - **RST sin cablear.** El driver hace un reset por hardware bajando y subiendo RST. Sin ese pin conectado a un GPIO, el chip queda dormido y devuelve `0x00` para siempre. No alcanza con atarlo a 3,3 V.

## 6.2 · Reed switch (estado de la puerta)

Un reed switch normalmente abierto con su imán en la hoja de la puerta. Una pata al `GPIO27`, la otra a GND. No lleva alimentación ni resistencia externa: el firmware activa el **pull-up interno** del ESP32.

| ESTADO FÍSICO | CONTACTO | LECTURA DEL GPIO | REPORTE |
|---|---|---|---|
| Puerta **cerrada** | imán cerca → cerrado | **`0`** | `estado_reed = CERRADO` |
| Puerta **abierta** | imán lejos → abierto | **`1`** | `estado_reed = ABIERTO` |

El firmware aplica un **antirrebote de 300 ms**: tiene que leer el mismo valor de forma sostenida antes de reportar un cambio. Sin eso, el golpe del cierre genera una ráfaga de eventos falsos.

> **NO USAR GPIO12 PARA EL REED**
>
> Se probó y hay que evitarlo: GPIO12 es un pin de *strapping* (MTDI) que **debe estar en LOW al arrancar**. Si el reed lo deja en alto durante el reset, la placa entra en bucle de arranque con el error `invalid header: 0xffffffff`. GPIO27 está verificado y funcionando.

## 6.3 · Detector de presencia DSC LC-100-PI

**No es un HC-SR501.** Es un PIR de alarma profesional cuya salida es un **relé de contacto seco normalmente cerrado**, y necesita alimentación de 12 V propia.

| | |
|---|---|
| Alimentación | 12 V de la fuente switching (mismo riel que el solenoide) |
| Salida | Contacto seco N.C. — terminales **C** y **NC** |
| Cableado | `C → GND · NC → GPIO16` |
| Modo del GPIO | Entrada con **PULL_UP interno** (obligatorio) |
| Calentamiento | ≈ 60 s desde que se energiza, antes de detectar bien |
| Duración del pulso | ≈ 1,7 s (tiempo de retención del relé interno) |

| SITUACIÓN | CONTACTO N.C. | LECTURA DEL GPIO16 |
|---|---|---|
| Reposo (sin movimiento) | cerrado → el pin queda atado a GND | **`0`** |
| Detección | abre → manda el pull-up interno | **`1`** |

**DOS COSAS QUE CUESTAN TIEMPO SI NO SE SABEN**

**El PULL_UP es obligatorio.** Sin él, al abrir el contacto la línea queda flotando y el «1» no es firme: se leen detecciones fantasma. Con pull-up, verificado en banco, los pulsos salen limpios.

**La sensibilidad se ajusta con el jumper de *pulse count*** (ponerlo en 2 para menos falsos positivos), **no con los trimpots** de la placa: esos son calibración de fábrica y moverlos descalibra el sensor.

El firmware limita los reportes a uno cada **30 segundos** (*throttle*). El nodo solo avisa que hubo movimiento; cuánto tiempo hace falta sin movimiento para cerrar una sesión lo decide el servidor.

**GPIO16 Y LA ELECCIÓN DEL MÓDULO**

El GPIO16 solo está libre en el **ESP32-WROOM**. En el **WROVER** lo usa la PSRAM interna y el PIR no funciona. Si en el maquetado se usa otra placa, hay que reubicar este pin en `config.py`.

### 6.4 · Solenoide y etapa de potencia

Es la parte del circuito donde un error se paga con componentes quemados. El solenoide es **fail-secure**: se energiza para *abrir*. Sin corriente queda trabado, que es exactamente el comportamiento deseado en un control de acceso.

| SITUACIÓN | GPIO15 | SOLENOIDE |
|---|---|---|
| **Reposo** (la mayor parte del tiempo) | `0 · LOW` | Des-energizado → puerta **trabada** |
| **Pulso de apertura** (800 ms) | `1 · HIGH` | Energizado → puerta **destrabada** |

Es lo que hace el firmware hoy, y es coherente con un MOSFET N en *low-side*. Ver §12: hay un comentario en el código que dice lo contrario y está pendiente de corregir.

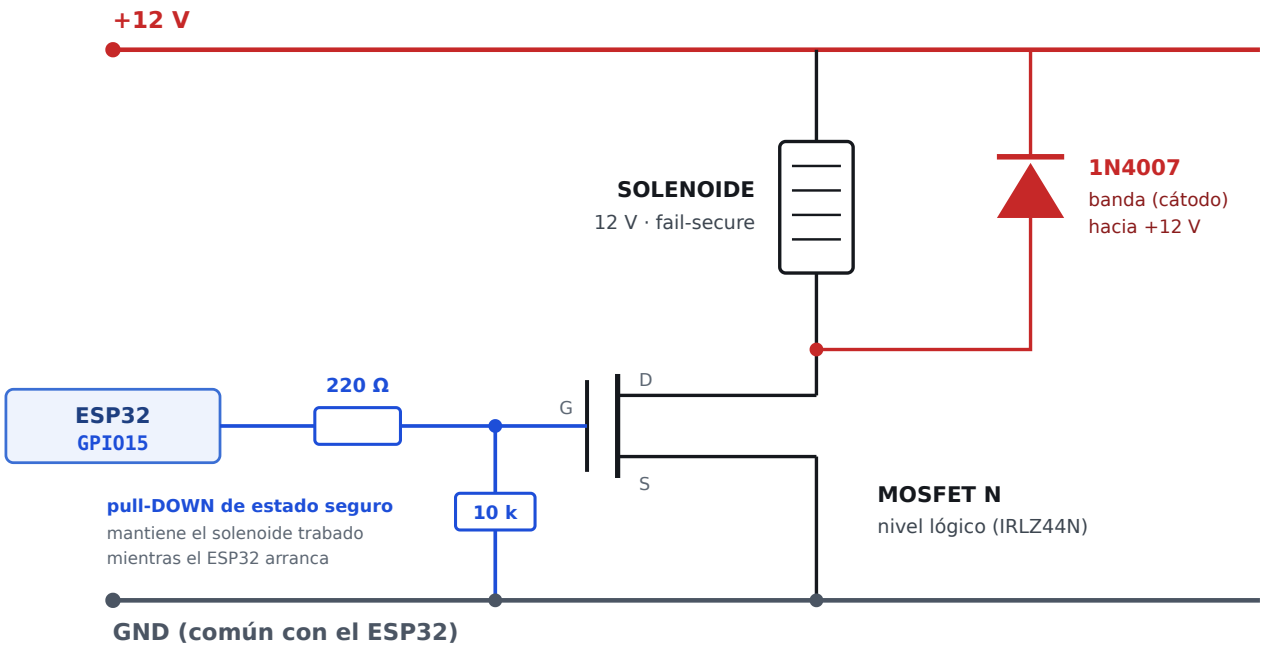



**Figura 3.** Etapa de potencia del solenoide. Tres elementos son obligatorios y ninguno es opcional: el **diodo 1N4007** en antiparalelo con la bobina (banda hacia +12 V), la resistencia de **10 kΩ a GND** en la compuerta, y la **masa común** entre los 12 V y el ESP32.

#### Por qué cada componente está ahí

| COMPONENTE | FUNCIÓN |
|---|---|
| **Diodo 1N4007** antiparalelo | Al cortar la corriente, la bobina genera un pico inverso de cientos de voltios. El diodo le da un camino para disiparse. **Sin él, el primer pulso puede destruir el MOSFET.** La banda del diodo (cátodo) va hacia +12 V. |
| **10 kΩ a GND** en la compuerta | Mantiene el MOSFET cortado mientras el ESP32 arranca o se resetea, cuando el GPIO todavía está flotando y no corre una sola línea de código. Es la garantía por *hardware* de que no hay pulsos espurios. |
| **220 Ω** en serie a la compuerta | Limita el pico de corriente que el GPIO entrega a la capacidad de compuerta al conmutar. Protege el pin del ESP32. |
| **MOSFET de nivel lógico** | El ESP32 entrega 3,3 V. Un MOSFET estándar (IRF540) necesita ~10 V para saturar: con 3,3 V queda a medio conducir, se calienta y el solenoide no tira. Hay que usar uno de nivel lógico. |

**GPIO15: un pin de strapping, y qué implica**

El GPIO15 es **MTDO**, uno de los pines de *strapping* del ESP32: tiene un pull-up interno y **arranca en ALTO** durante el reset, antes de que corra código. Como el firmware energiza el solenoide con nivel ALTO, ese estado inicial **no es seguro por sí solo**: por eso la resistencia externa de **10 kΩ a GND** no es un adorno, es la que gana contra el pull-up interno (~45 kΩ) y deja la compuerta por debajo del umbral de conducción durante el arranque.

> **VERIFICAR EN LA MAQUETA ANTES DE CONECTAR EL SOLENOIDE**
>
> Antes de cablear la bobina, medir con el téster la tensión en la compuerta del MOSFET durante un reset de la placa. **Tiene que quedar por debajo de ~1 V.** Si sube a 3,3 V al arrancar, falta la resistencia de 10 kΩ o está mal puesta, y el solenoide va a dispararse en cada reinicio.

## 07 Reglas de uso de los GPIO

Media docena de reglas del ESP32 que, si no se conocen, producen fallos que parecen bugs de software y no lo son. Vale la pena leerlas antes de reubicar cualquier pin en la maqueta.

| PINES | RESTRICCIÓN | QUÉ PASA SI SE IGNORA |
|---|---|---|
| **GPIO6–11** | **Prohibidos** | Están conectados a la memoria flash SPI del módulo. Usarlos cuelga la placa de forma irrecuperable. |
| **GPIO34–39** | Solo entrada | No pueden ser salida y **no tienen pull-up/pull-down internos**. Para el reed o el PIR habría que agregar resistencias externas. |
| **GPIO12** | Strapping (MTDI) debe estar en LOW al arrancar | Bucle de arranque con `invalid header: 0xffffffff`. Ya nos pasó con el reed acá. |
| **GPIO15** | Strapping (MTDO) arranca en ALTO | Cualquier carga conectada recibe un pulso durante el arranque. Por eso lleva la resistencia de 10 kΩ a GND (§6.4). |
| **GPIO0 · GPIO2** | Strapping (modo de arranque) | Si algo los fuerza a un nivel raro durante el reset, la placa entra en modo de descarga y no arranca el firmware. GPIO2 lleva el LED integrado, que es carga liviana y no molesta. |
| **GPIO5** | Strapping (idle en alto) | Sale en alto al arrancar. Para un *chip select* activo en LOW —que es nuestro caso— eso es justo lo que se quiere: el RC522 arranca deseleccionado. |
| **GPIO16 · GPIO17** | Ocupados en WROVER | Los usa la PSRAM. El PIR (GPIO16) no funciona si se maqueta con un WROVER en vez de un WROOM. |

### Corriente por pin

Cada GPIO del ESP32 entrega cómodamente hasta **12 mA** (el máximo absoluto es 40 mA, pero no conviene acercarse). Los LEDs van con resistencia de 220–330 Ω, lo que da unos 5–7 mA: sobrado para verlos y muy lejos del límite. **Ninguna carga de potencia se conecta directamente a un GPIO** — para eso está el MOSFET.

### Si hace falta mover un pin

Todo el mapa vive en un solo archivo, `firmware/nodo_panol/config.py`. No hay números de pin dispersos por el código. Cambiar un pin es editar una línea de ese archivo y volver a cargar el firmware; no hay que tocar la lógica.

> **PROBAR DE A UN SENSOR POR VEZ**
>
> El mismo archivo tiene un diccionario SENSORES con tres banderas (`rfid`, `reed`, `pir`). Un sensor puesto en `False` ni siquiera se instancia: no se lee, no reporta y no ensucia la consola. Es la forma prevista de ir armando la maqueta de a partes sin pelearse con eventos fantasma de lo que todavía no está cableado.

## 08 Lógica: la máquina de estados

El nodo tiene cuatro estados y vive en RAM. Si se corta la luz, esta máquina se pierde — y no importa: al arrancar nace en `IDLE`, que es el estado seguro, y vuelve a reportar.

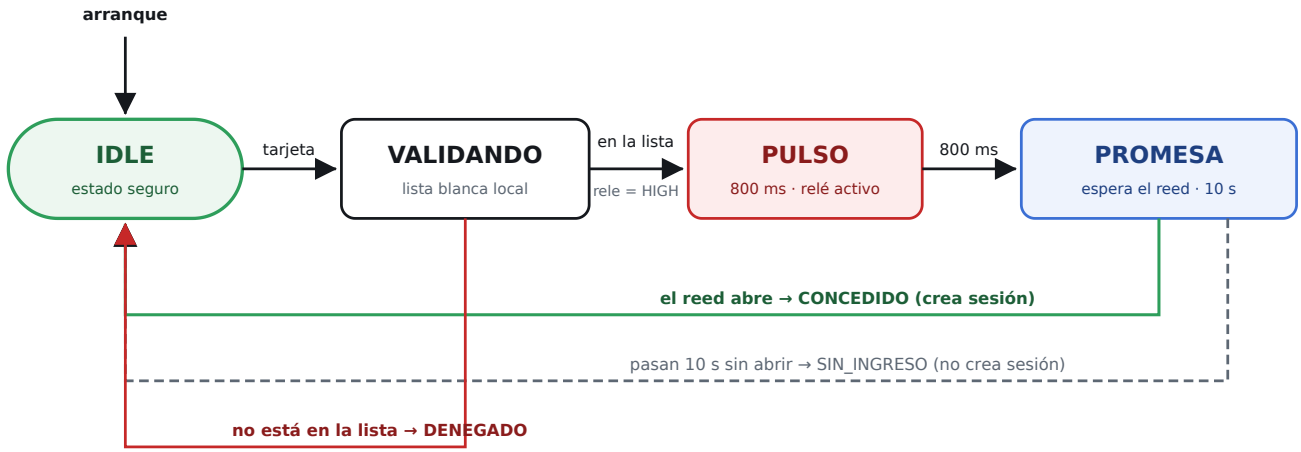



**Figura 4.** Máquina de estados del nodo. Los eventos del reed y del PIR se reportan **siempre**, en cualquier estado, en paralelo a esta máquina: no participan de las transiciones salvo por el flanco de apertura que cierra la promesa.

### La «promesa»: por qué no alcanza con pulsar el solenoide

Que la tarjeta sea válida no significa que alguien haya entrado. Puede pasar la tarjeta, ver que destraba y arrepentirse. Por eso el nodo hace una **promesa**: después del pulso espera hasta 10 segundos a que el reed confirme que la puerta realmente se abrió.

- **El reed abre dentro de la ventana** → se reporta `CONCEDIDO`, y el servidor **crea una sesión** a nombre de esa tarjeta.
- **Se cumplen los 10 s sin abrir** → se reporta `SIN_INGRESO`. Queda registrado que la puerta se destrabó, pero **no se abre sesión**.

La ventana arranca al comienzo del pulso, no al final: si alguien está esperando del otro lado y abre durante esos 800 ms, ese flanco también cuenta.

> **DETALLE DE IMPLEMENTACIÓN QUE IMPORTA PARA EL MAQUETADO**
>
> El pulso **no es bloqueante**: no se hace con un `sleep(800)`. Si lo fuera, el nodo quedaría ciego al reed justo en la ventana donde tiene que vigilarlo. El bucle principal corre cada 50 ms y va contando el tiempo transcurrido.
>
> Por el mismo motivo, **el trabajo de red solo se hace en `IDLE`**: un POST puede tardar hasta 3 segundos, y ese tiempo no puede caer en medio del pulso ni de la promesa.

## Cronograma de una apertura exitosa




**Figura 5.** Secuencia temporal real. El evento `acceso CONCEDIDO` no se emite al validar la tarjeta, sino recién cuando el reed confirma la apertura. Si el reed nunca cambia, a los 10 s se emite `SIN_INGRESO` en su lugar.

## Tabla de tiempos

| PARÁMETRO | VALOR | QUÉ CONTROLA |
|---|---|---|
| `T_PULSO_SOLENOIDE_MS` | **800 ms** | Duración del pulso sobre el solenoide. |
| `T_APERTURA_MS` | **10 s** | Ventana de la promesa: desde el inicio del pulso hasta que expira. |
| `T_ANTIRREBOTE_REED_MS` | **300 ms** | Estabilidad mínima antes de reportar un cambio del reed. |
| `THROTTLE_PIR_MS` | **30 s** | Máximo un reporte de movimiento cada 30 segundos. |
| `T_MISMA_TARJETA_MS` | **2 s** | Ignora la misma tarjeta leída de nuevo dentro de ese lapso. |
| `T_CICLO_MS` | **50 ms** | Período del bucle principal. |
| `T_WDT_MS` | **8 s** | Watchdog: reinicia el nodo si el firmware se cuelga. **Desactivado en banco** (molesta al trabajar desde el REPL); se activa al montar en la puerta. |
| `T_HEARTBEAT_MS` | **60 s** | Señal de vida hacia el servidor. |
| `T_REFRESCO_WHITELIST_MS` | **15 min** | Cada cuánto baja la lista blanca del servidor. |

## Alta de tarjetas sin servidor

Hay una función que no está en la especificación pero es la que hace usable el pañol: mantener apretado el **botón del GPIO32** y acercar una tarjeta la **da de alta** en vez de abrir la puerta. Queda guardada en `uids.txt` en la flash del nodo.

> **UN MODO DE FALLA QUE VALE LA PENA ENTENDER**
>
> El botón tiene **prioridad sobre abrir la puerta**. Si quedara trabado en «apretado», el nodo daría de alta cada tarjeta en silencio y **nunca abriría** — con gente quedándose afuera sin entender por qué.
>
> Por eso el firmware lo comprueba al arrancar: en reposo el pin **debe leer 0**. Si lee 1, desactiva la función de grabado, avisa por consola y sigue funcionando como lectora normal. Es preferible perder el alta de tarjetas antes que perder la apertura. En la maqueta, esto significa que **el pull-down del botón tiene que estar bien puesto** o el grabado no va a funcionar nunca.

## 09 Qué reporta el nodo

Tres tipos de evento y un latido. Cada evento viaja con un identificador único para que el servidor pueda descartar duplicados si hubo un reenvío.

| TIPO | DATOS | CUÁNDO SE EMITE |
|---|---|---|
| **acceso** | `uid_hex · resultado · modo_degradado` | `CONCEDIDO` (el reed confirmó) · `SIN_INGRESO` (expiró la promesa) · `DENEGADO` (no está en la lista) |
| **puerta** | `estado_reed · inicial` | Cada cambio estable del reed. También una vez al arrancar, con `inicial=true`, para que el servidor sincronice sin recordar nada previo. |
| **pir** | — | Movimiento detectado, como máximo una vez cada 30 s. |
| **heartbeat** | `uptime · rssi · modo_degradado` | Cada 60 s. Su ausencia durante 5 minutos dispara una alarma en el servidor. |

**Estructura de un evento**

```
{
  "event_id": "panol-lab01-puerta-7-42",  // nodo + nº de arranque + secuencia
  "ubicacion_id": "panol-lab01",
  "nodo_id": "panol-lab01-puerta",
  "tipo": "acceso",
  "timestamp": "2026-08-19T14:32:05-03:00",  // null si el nodo no está en hora
  "uptime_ms": 184320,
  "datos": { "uid_hex": "DE:AD:BE:EF", "resultado": "CONCEDIDO", "modo_degradado": false }
}
```

### Red

| | |
|---|---|
| Transporte | HTTPS hacia la API del servidor (el broker MQTT lo habla el servidor, no el nodo) |
| WiFi | Multi-red: escanea el aire, elige la conocida con mejor señal y prueba la siguiente si falla |
| Credenciales | En `secrets.py`, **fuera del repositorio** |
| Reintento de asociación | Cada 30 s si se cae el AP, sin bloquear el bucle principal |
| Hora | NTP al arrancar y resincronización cada 6 h (el ESP32 no tiene reloj con pila) |
| Timeout HTTP | 3 s — corto a propósito: la red nunca debe demorar la puerta |

### Modo degradado

Cuando el nodo no logra hablar con el servidor entra en **modo degradado**. Conviene entender qué significa exactamente, porque no es lo que suena: **no** quiere decir «no puedo decidir». El nodo decide igual, contra su lista blanca local. Significa «no puedo refrescar la lista ni reportar ahora mismo». Los eventos se acumulan en `cola.log`, en la flash, y se envían cuando vuelve la red.

| QUÉ SE CORTA | PUERTA (DESDE AFUERA) | AUDITORÍA | RECUPERACIÓN |
|---|---|---|---|
| Solo el nodo | Trabada (fail-secure) | Eventos en `cola.log` | Automática al arrancar |
| Solo el servidor | **Sigue abriendo normal** | El nodo encola | Automática al reiniciar |
| WiFi / red | **Sigue abriendo normal** | El nodo encola con su timestamp | Automática al reconectar |
| Todo (apagón) | Trabada | Hueco visible por falta de latidos | Nodo → IDLE, servidor → base |

En todos los casos, la **evacuación desde adentro está garantizada por el picaporte mecánico**, que no depende de nada eléctrico.

La cola tiene un tope de **5000 eventos**. Si se llena —un corte de red de días— se descartan los **nuevos** y se conserva la evidencia más vieja, que es la del comienzo del incidente.

## 10 Armado paso a paso

Orden sugerido para el maquetado. La idea es que en cada paso haya **una sola cosa nueva** que pueda estar mal, y que se pueda verificar antes de seguir.

### Paso 1 · La placa sola

- [ ] ESP32-WROOM alimentado por USB, con MicroPython cargado.
- [ ] Verificar que arranca y llega al REPL sin bucles de reinicio.
- [ ] Poner los tres sensores en `False` en SENSORES y `USAR_RED = False`.

### Paso 2 · Reed switch

- [ ] Una pata a GPIO27, la otra a GND. Sin resistencias externas.
- [ ] Activar `"reed": True`.
- [ ] Acercar y alejar el imán: la consola tiene que mostrar `REED ABIERTA` / `CERRADA` y el LED del GPIO33 seguirlo.

### Paso 3 · Lector RC522

- [ ] Cablear los cinco pines. **Confirmar VCC en 3,3 V con el téster antes de energizar.**
- [ ] Activar `"rfid": True`.
- [ ] Al arrancar tiene que aparecer `RC522 OK (reg 0x37 = 0x92)`. Si dice «no responde», revisar MISO y RST (§6.1).
- [ ] Acercar una tarjeta: debe imprimir el UID.

### Paso 4 · PIR (ya con 12 V en la maqueta)

- [ ] Alimentar el PIR con los 12 V y **unir la masa de la fuente al GND del ESP32**.
- [ ] C → GND, NC → GPIO16.
- [ ] Esperar el minuto de calentamiento antes de juzgar el comportamiento.
- [ ] Activar `"pir": True` y verificar que reporta movimiento.

### Paso 5 · Etapa de potencia — sin la bobina todavía

- [ ] Armar MOSFET, resistencia de compuerta de 220 Ω y pull-down de 10 kΩ (figura 3).
- [ ] **Con el téster en la compuerta, resetear la placa.** Durante el arranque la tensión tiene que quedar por debajo de ~1 V.
- [ ] Recién si eso se cumple, conectar el solenoide y el diodo 1N4007 (banda hacia +12 V).

### Paso 6 · Prueba del solenoide, aislada

Hay un script que pulsa el solenoide **sin la máquina de estados ni tarjetas**, para separar un problema eléctrico de un problema de lógica:

```
.venv/bin/mpremote connect /dev/ttyUSB0 run firmware/nodo_panol/prueba_solenoide.py
```

No escribe en la flash ni toca `main.py`. Antes de correrlo, ver la advertencia de §12.

### Paso 7 · Integración

- [ ] Los tres sensores en `True` y el solenoide conectado.

- [ ] Pasar una tarjeta autorizada: debe verse la secuencia completa `IDLE → VALIDANDO → PULSO → PROMESA → IDLE` con resultado `CONCEDIDO`.
- [ ] Pasar una tarjeta desconocida: `DENEGADO`, sin pulso.
- [ ] Pasar una autorizada y **no abrir**: a los 10 s, `SIN_INGRESO`.
- [ ] Recién entonces, poner `USAR_RED = True`.

### Herramientas de diagnóstico disponibles

| SCRIPT | PARA QUÉ SIRVE |
|---|---|
| `prueba_sensores.py` | Lee los tres sensores en crudo, sin FSM. |
| `escanear_gpio.py` | Descubre en qué pin cae cada cosa cuando el cableado no está claro. |
| `probe_rc522.py` | Machaca el registro `0x37` para distinguir un lector muerto de uno intermitente. |
| `debug_pir.py` | PIR en crudo, sin el throttle de 30 s. Sirve para ver si hay rebotes. |
| `prueba_solenoide.py` | Pulso del MOSFET aislado de la lógica. |

Ninguno escribe en la flash. Se corren con `mpremote ... run` o abriéndolos en Thonny con F5.

> **NOTA SOBRE EL PUERTO SERIE**
>
> El ESP32 aparece como `/dev/ttyUSB0`. Thonny suele quedarse con el puerto tomado: hay que hacer *Run → Disconnect* antes de usar `mpremote`. Si `mpremote` falla con «could not enter raw repl», normalmente es porque el firmware está en su bucle de verificación del RC522 de 20 s — esperar y reintentar.

## 11 Estado verificado del prototipo

Qué está probado sobre hardware real y qué no. Vale la pena tenerlo claro antes de copiar el diseño a una maqueta.

| SUBSISTEMA | ESTADO | DETALLE DE LA VERIFICACIÓN |
|---|---|---|
| **RC522 (RFID)** | VERIFICADO | Registro `0x37 = 0x92`, lee tarjetas y valida contra la lista. Costó llegar: primero daba `0x00`/`0xFF` por MISO y por el RST sin cablear. |
| **Reed switch** | VERIFICADO | Funcionando en GPIO27 con pull-up interno. Se probó GPIO13 y se volvió a 27. GPIO12 descartado por strapping. |
| **PIR DSC LC-100-PI** | VERIFICADO | Pulsos limpios y consistentes de ~1,7 s, sin rebotes, con masa firme y pull-up interno. |
| **FSM completa** | VERIFICADO | Secuencia `IDLE → VALIDANDO → PULSO → PROMESA → IDLE` con `CONCEDIDO` confirmado por el reed real. |
| **Red y reporte** | VERIFICADO | El nodo, desde otra red, reporta al servidor por HTTPS y crea sesiones con nombre en la base (`modo_degradado=false`). |
| **Botón GPIO32** | PARCIAL | Leyó valores inconsistentes entre arranques. Por eso existe la protección descrita en §8. En la maqueta, poner un pull-down bien firme. |
| **Solenoide + MOSFET** | A CONFIRMAR | Es la parte menos ejercitada del conjunto y tiene una inconsistencia pendiente. Ver §12. |

> **LO IMPORTANTE PARA EL MAQUETADO**
>
> Toda la parte de sensores y lógica está probada y se puede copiar con confianza. La única zona que hay que **verificar con téster antes de dar por buena** es la etapa de potencia del solenoide.

## 12 Puntos abiertos y advertencias

Honestidad sobre lo que todavía no está cerrado. Son tres cosas, y las tres afectan la misma parte del circuito.

### 12.1 · Los comentarios del código sobre el relé contradicen al código

En `config.py`, los comentarios que acompañan a `PIN_RELE` y `RELE_ACTIVO_EN` afirman que el montaje es **activo en LOW** con reposo en HIGH. **El código hace exactamente lo contrario:**

| FUENTE | REPOSO | ACTIVACIÓN |
|---|---|---|
| Comentarios de `config.py` | HIGH | LOW |
| **Lo que ejecuta `main.py`** | **LOW** (`1 − 1 = 0`) | **HIGH** (`RELE_ACTIVO_EN = 1`) |

Lo que corre en la placa es **activo en HIGH**, que además es lo coherente con un MOSFET N en *low-side*: compuerta en alto → conduce → solenoide energizado. La tabla de §6.4 y la figura 3 de este documento describen **el comportamiento real del firmware**, no los comentarios.

> **CONSECUENCIA PRÁCTICA**
>
> El mismo comentario dice que hace falta un **pull-up** externo de 10 kΩ. Eso sería correcto para un montaje activo-LOW, pero con el firmware actual **lo que hay que poner es un pull-DOWN**. Un pull-up dejaría la compuerta alta en reposo y el solenoide energizado de forma permanente. El script `prueba_solenoide.py` lo dice bien («pull-DOWN de 10k»); el comentario de `config.py` quedó viejo.

### 12.2 · `prueba_solenoide.py` no pulsa

El script de prueba aislada calcula el estado de reposo como `0 − RELE_ACTIVO_EN`, mientras que `main.py` usa `1 − RELE_ACTIVO_EN`. La diferencia no es cosmética: con esa fórmula el valor de reposo y el de activación **terminan siendo el mismo nivel lógico**, así que el script nunca genera un flanco.

| CONFIGURACIÓN | «REPOSO» CALCULADO | «ACTIVO» | RESULTADO |
|---|---|---|---|
| `RELE_ACTIVO_EN = 1` | `−1 → HIGH` | `1 → HIGH` | Solenoide **siempre energizado**, incluso al terminar el script |
| `RELE_ACTIVO_EN = 0` | `0 → LOW` | `0 → LOW` | Solenoide **nunca energizado**: no pasa nada |

> **ANTES DE CORRER EL SCRIPT EN LA MAQUETA**
>
> Corregir la línea 22 de `prueba_solenoide.py`, cambiando `REPOSO = 0 - config.RELE_ACTIVO_EN` por `REPOSO = 1 - config.RELE_ACTIVO_EN`, que es la fórmula que usa `main.py`.
>
> Sin esa corrección, con la configuración actual el solenoide queda **energizado de forma continua** durante toda la prueba y también después de terminar — la bobina se calienta y se puede dañar. Muy probablemente sea lo que produjo la impresión de «lógica invertida» que quedó anotada en los comentarios del punto 12.1.

### 12.3 · Documentación desactualizada en el repositorio

| DÓNDE | DICE | LO CORRECTO |
|---|---|---|
| `README.md` · mapa de pines | Relé en GPIO26 | **GPIO15** |
| `README.md` · mapa de pines | PIR HC-SR501, jumper retrigger H | **DSC LC-100-PI**, contacto seco N.C. a 12 V |
| `README.md` · mapa de pines | «SoftSPI» | SPI de **hardware** (SPI2/VSPI) |
| `main.py` · `poll_pir()` | Comentario sobre el HC-SR501 y su jumper | Comentario heredado; el sensor real es el DSC |

### 12.4 · Decisiones que dependen del montaje final

- **Watchdog desactivado.** `USAR_WDT = False` mientras se trabaja en banco, porque cortar con Ctrl+C y quedarse en el REPL dispara el reinicio. **Hay que activarlo al montar en la puerta.**
- **Módulo de relé en lugar de MOSFET.** Si en la maqueta se usa un módulo de relé optoacoplado —la mayoría son activos en LOW— hay que cambiar `RELE_ACTIVO_EN` a `0` y sustituir el pull-down por un pull-up. El firmware ya está preparado para las dos polaridades: es una sola constante.
- **Credenciales por defecto.** Las claves `cambiar-*` del broker MQTT son conocidas a propósito y hay que rotarlas antes de instalar en el colegio.

---

**Fuentes de este documento.** Firmware: `firmware/nodo_panol/{config,main,red,cola,reloj,mfrc522,prueba_solenoide}.py`. Diseño: `docs/DISENO.md`. Verificaciones sobre hardware real registradas en las notas de banco del proyecto. Los valores de pines y tiempos se leyeron del código, no de la documentación previa.